

AI Declaration Form

1.

I acknowledge the use of [1] ChatGPT [2] (<https://chatgpt.com/c/67cff7f5-8d60-8012-9462-bc7a1edb8c23>) to [3] generate materials that were [4] included within my final assessment in modified form. I entered the following prompts on 23 March 2025:

[3] Can this be coded to be playable according to Satorini gameplay? after that help me to make an executable file so it's playable on a window.

I acknowledge the use of [1] ChatGPT [2] (<https://chatgpt.com/c/67cff7f5-8d60-8012-9462-bc7a1edb8c23>) [3] to refine the academic language and accuracy of my own work. On 23 March 2025, I entered the following prompt [3]. [4] The output (here) [5] was then modified further to better represent my own tone and style of writing.

[3]

Can this be coded to be playable according to Satorini gameplay? after that help me to make an executable file so it's playable on a window.

[4]

Yes! To make this Santorini board playable, we need to implement the following game mechanics:

- ◆ Steps to Make the Game Playable

Allow players to select and move their workers.

Enforce movement rules (e.g., only move to adjacent tiles, can move up by 1 level, etc.).

Enable building construction after moving.

Implement win/loss conditions (win by reaching level 3, lose if no valid moves).

Add a turn system (players take turns moving and building).

Once the game logic is complete, you can create an executable .jar file.

☒ 1 Compile the Game

Open the terminal and navigate to the folder containing SantoriniGame.java, then run:

```
javac SantoriniGame.java
```

② Create a Manifest File (manifest.txt)

Create a text file named manifest.txt with this content:

Main-Class: jframe.SantoriniGame

③ Package the JAR

Run this command in the terminal:

```
jar cfm SantoriniGame.jar manifest.txt SantoriniGame.class
```

④ Run the Executable JAR

Now, you can launch the game with:

```
java -jar SantoriniGame.jar
```

Now you have a fully playable Santorini game in Java Swing with an executable JAR file!

 Let me know if you need more features! 😊

2.

I acknowledge the use of [1] ChatGPT [2] (<https://chat.openai.com/>) [3] to refine the academic language and accuracy of my own work. On 25 March 2025 I entered the following prompt [3]. [4] The output (here) [5] was then modified further to better represent my own tone and style of writing.

[3]

how do i create an executable file for my application on MacOS?

[4]

Step 1: Compile the main class used.

- Navigate to the project root. (FIT3077Santorini in my case).
- Run the code.

```
javac -d out -cp src src/prototype/pee_yee_peen/Application.java
```

- The compiled file will now be compiled and stored in the 'out' directory.

Step 2: Create a JAR File

```
jar cfe FIT3077Santorini.jar prototype.pee_yee_peen.Application -C out .
```

Step 3: Check the functionality of the JAR file.

```
java -jar FIT3077Santorini.jar
```

```
jpackage --name FIT3077Santorini \  
  --input out \  
  --main-jar FIT3077Santorini.jar \  
  --main-class prototype.pee_yee_peen.Application \  
  --type app-image
```

Method 1: Using jpackage (Recommended for Java 14+)

jpackage is a built-in tool from Java 14+ that can generate a .app bundle, making your application feel like a native macOS app.

Steps:

1. Ensure Java 14 or later is installed

```
sh
```

CopyEdit

```
java --version
```

If not installed, use Homebrew:

```
sh
```

CopyEdit

```
brew install openjdk
```

2. Run jpackage to create a .app bundle

```
sh
```

CopyEdit

```
jpackage \  
  --name MyApp \  
  --input . \  
  --main-jar MyApp.jar \  
  --main-class com.example.Main \  
  --type app-image \  
  --icon app-icon.icns \  
  --app-version 1.0 \  
  --dest output
```

- o --name MyApp → Name of the .app file
- o --main-jar MyApp.jar → Your main JAR file
- o --main-class com.example.Main → Main class (update accordingly)
- o --type app-image → Creates a .app bundle
- o --icon app-icon.icns → Application icon (optional)
- o --app-version 1.0 → Version of the app
- o --dest output → Output folder

3. Run your .app

```
sh
```

CopyEdit

```
open output/MyApp.app
```

Method 2: Creating a Shell Script (Simpler Alternative)

If you just need a quick executable file, you can use a shell script.

Steps:

1. Create a script file (MyApp.command)

Inside your project folder, create a file named MyApp.command and add the following:

```
sh
```

```
CopyEdit
#!/bin/bash
cd "$(dirname "$0")"
java -jar MyApp.jar
2.      Make the script executable
```

```
sh
CopyEdit
chmod +x MyApp.command
3.      Run it
Double-click MyApp.command, or run:
sh
CopyEdit
./MyApp.command
```

Method 3: Using Platypus (For GUI Apps)

Platypus wraps your script into a native .app.

Steps:

```
1.      Install Platypus
sh
CopyEdit
brew install --cask platypus
2.      Create a new application
o       Open Platypus
o       Set Script Path to a new script (run.sh) containing:
```

```
sh
CopyEdit
#!/bin/bash
cd "$(dirname "$0")"
java -jar MyApp.jar
o       Add MyApp.jar under "Bundled Files"
o       Click "Create App"
3.      Run your new .app
Double-click MyApp.app to launch your app.
```

Legend: [1] AI system(s); [2] link to the tool; [3] how the tool was used or prompts included; [4] output included in the task [5] edits and changes made to the output.